Optimization Beyond Plain SGD

Advanced Machine Learning

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

Learning objectives

- Feel *why* plain SGD can stall, before seeing any fix
- Sort every fix into one of two buckets: tune the step size, or tune what we compute from the gradient
- Read Adam's update rule as "momentum and Adagrad, combined" – and know where that combination gets messy
- Recognize the canonical Adam + scheduler training loop as a small edit to the loop already written by hand

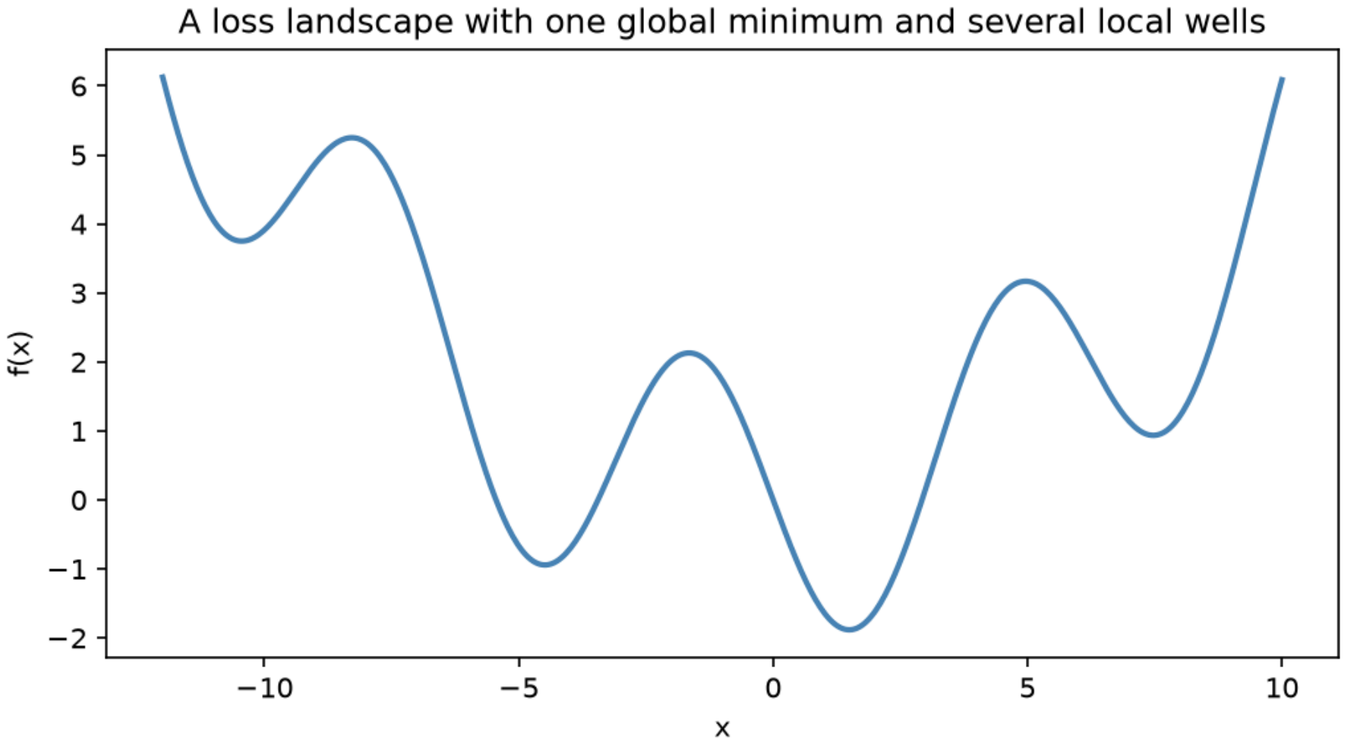
Why plain SGD gets stuck

Same loop, a harder landscape

Where we left off

- Every epoch: pull a mini-batch from a data loader, run the forward pass, compute the loss
- `loss.backward()` computes the gradient; `optimizer.step()` applies it
- So far, `optimizer` has always meant plain `torch.optim.SGD`, with one fixed learning rate for the whole run

A harder landscape



Real loss surfaces aren’t smooth bowls – they have a rough overall trend *and* bumps along the way

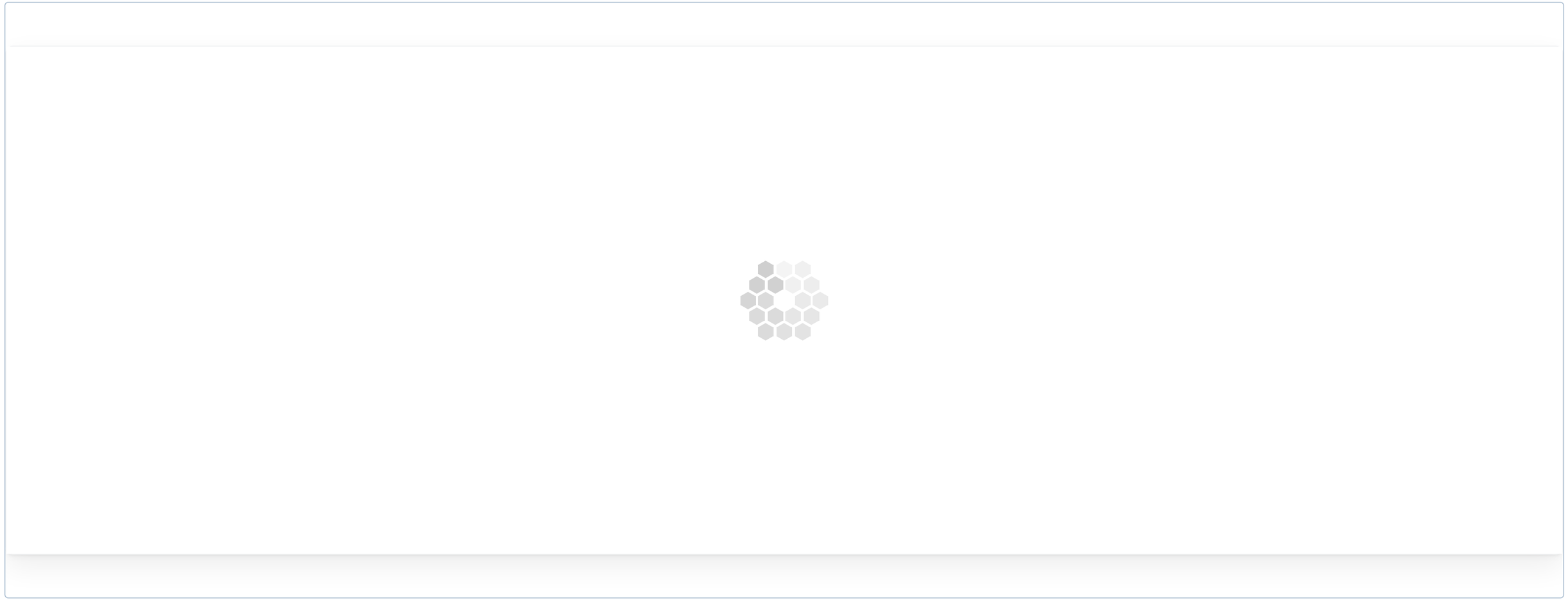
- Especially once we have multiple layers!

Plain gradient descent, as code

```
1 def step__gradient_descent(x, grad, eta):
2     return x - eta * grad
```

- `x`: the current parameter value
- `grad`: the gradient of the loss with respect to `x`, at this step
- `eta`: the learning rate – one fixed number, for the whole run
- Every optimizer for the rest of this lecture is a variation on this exact function – same shape, one more ingredient at a time

Watch it stall



- It heads downhill, then **stops** – not because the loss is minimized, but because the *local* gradient runs out before the global minimum is reached

Two knobs we can turn

Two different kinds of fix

The Big Idea

$w \leftarrow w - \eta \frac{\partial L}{\partial w}$ has two parts:

- η: our learning rate
- $\frac{\partial L}{\partial w}$: Our gradient step calculation

We can tweak these to get better performance!

The two buckets

Tune the step size itself

- The learning rate η doesn't have to be one fixed number
- Shrink, grow, or otherwise schedule it as training goes on

Tune what we compute from ∇L

- Instead of stepping on the raw gradient, use a running summary of it
- Carry momentum forward, or scale each parameter's step by its own gradient history

Learning rate scheduling

Let η change over time

Decaying the learning rate

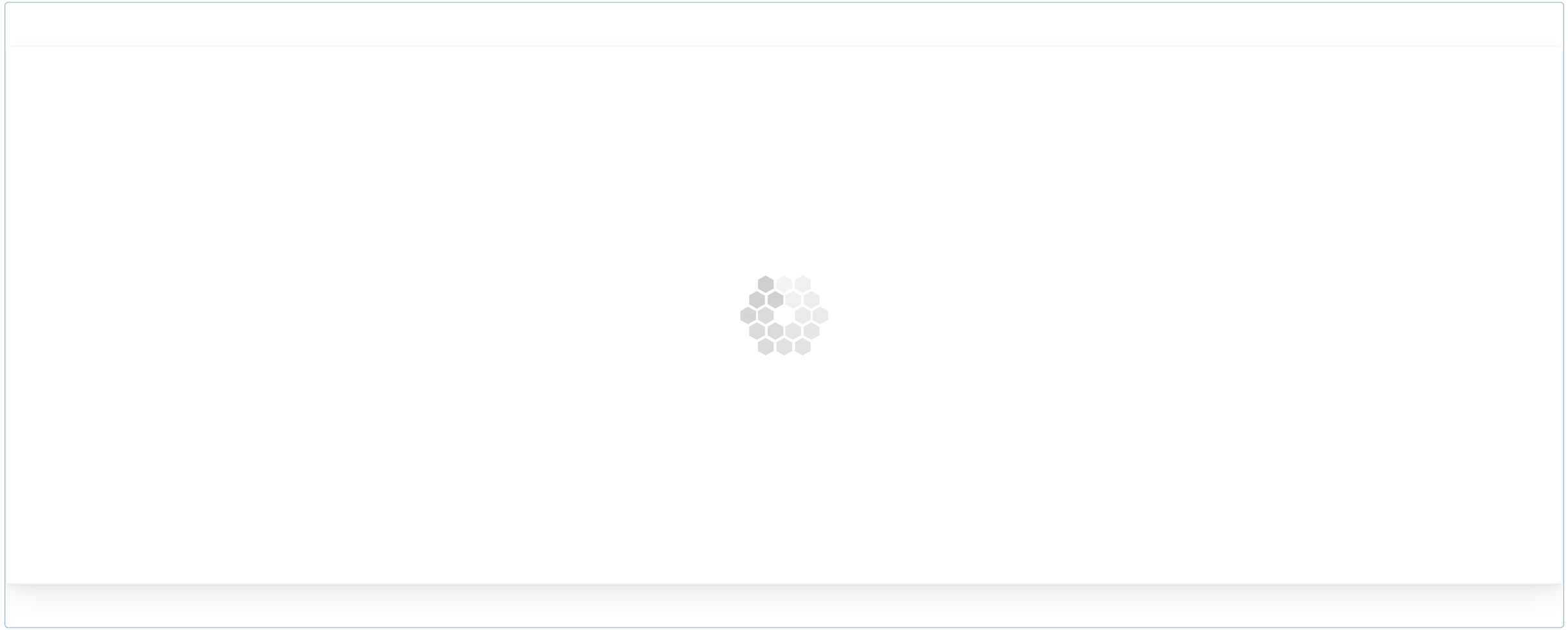
$$\eta_t = \eta_0 \cdot \gamma^t \qquad (\gamma < 1)$$

```
1 def scheduled_lr(eta_0, gamma, t):
2     return eta_0 * gamma ** t
```

- η_0: the starting learning rate – large enough to clear early obstacles
- γ: the decay factor – how much of the previous step size survives each step
- t: the step count – the update itself is still `step__gradient_descent`, just called with `eta = scheduled_lr(eta_0, gamma, t)` instead of one fixed number

The idea is that we decrease the step over time

See it work



- A large η_0 clears the well that stalled the constant-η run earlier
- Decay settles it down into a local (hopefully global) minima

Adagrad

Scale the step by how much a parameter has already moved

The key insight

Remember how in Gradient Descent with $\hat{y} = wx + b$ we saw big changes in w but slow changes in b?

w and b often need different amounts of updating

- A parameter whose gradient has been small likely hasn't found its value yet
  - It should have a big step to get it where it is going
- A parameter who has had big gradients has moved a lot
  - That sounds like a job for a learning rate reduction so it doesn't overshoot

The math behind it

$$s \leftarrow s + g^2 \qquad x \leftarrow x - \frac{\eta}{\sqrt{s} + \epsilon} g$$

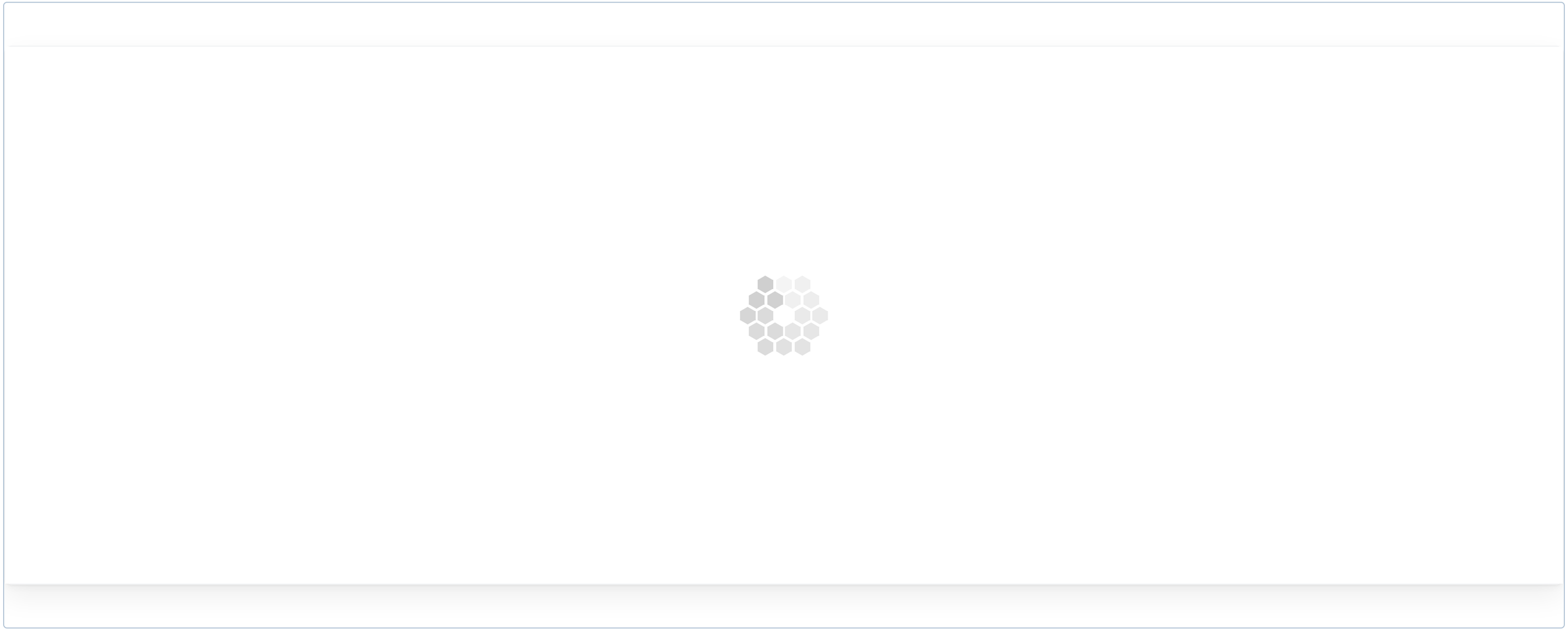
- g: the gradient of the loss with respect to *this one parameter*, at this step
  - e.g. $\frac{\delta L}{\delta w_0}$
- s: a running sum of this parameter's squared gradients so far – grows every step, never shrinks
- ϵ a small number to make sure we don't divide by 0 while s is near to 0

Dividing η by $\sqrt{s}$ makes step sizes shrink as we apply more gradients

```python
def step__adagrad(x, s, grad, eta, eps=1e-8):
    s = s + grad ** 2
    return x - eta / (s**0.5 + eps) * grad, s
```

- Same shape as `step__gradient_descent` – the only change is `eta` becomes `eta / (s**0.5 + eps)`, a different effective learning rate for every parameter

See it work



> **Note**
>
> This is basically just GD right now – it needs several parameters really work

Momentum

Keep a memory of where we were heading

Carrying velocity forward

$$v \leftarrow \beta v - \eta \nabla L(x) \qquad x \leftarrow x + v$$

```
1 def step__momentum(x, v, grad, eta, beta):
2     v = beta * v - eta * grad
3     return x + v, v
```

- β : how much of the previous step's velocity survives into this one
  - Will be in the range (0, 1]
- Gradient Descent with momentum behaves like a rolling ball
  - If the gradient keeps pointing the same general direction, v gets bigger
  - If the gradient reverses, momentum still carries some of the old direction forward for a step or two, instead of stopping instantly

See it work

Adam

Momentum and Adagrad, combined

What if we did both?

Adagrad (and learning rate adjustment)

- Scales the learning rate at each step
- Ensures we don’t overshoot late in training

Momenum

- Keeps the gradient moving in the same direction
- Allows us to skip out of local maxima

Adam combines these:

- Track *where* the gradient has been pointing (momentum’s idea)
- Track *how much* it’s moved so far (Adagrad’s idea)

Adam: Updates

$$m \leftarrow \beta_1 m + (1 - \beta_1) g$$

$$s \leftarrow \beta_2 s + (1 - \beta_2) g^2$$

$$x \leftarrow x - \frac{\eta}{\sqrt{s} + \epsilon} m$$

The same equations, as code

```
1 def step__adam(x, m, s, grad, eta, beta1=0.9, beta2=0.999, eps=1e-8):
2     m = beta1 * m + (1 - beta1) * grad
3     s = beta2 * s + (1 - beta2) * grad ** 2
4     return x - eta * m / (s**0.5 + eps), m, s
```

- Line 1: `step__momentum`'s update, renamed
- Line 2: `step__adagrad`'s update, renamed and averaged instead of summed
- Line 3: the same shape as `step__gradient_descent`
  - `grad` is replaced by `m`
  - `eta` is replaced by `eta / (s**0.5 + eps)`

What Adam is actually doing

Instead of *just* tuning the step size *or* the gradient, `Adam` does both

- It makes use of momentum to keep the gradient moving over local maxima
- It scales the learning rate *per parameter every step* so we don't move too much

Adam works well by using both at once, not just choosing one

See it work

Summary

Put it all together

Study guide: optimizers

optimizer	core idea	new knob	useful when
SGD	move away from gradient	η	the baseline
+ Scheduling	change η over time	η_0, γ	honing in on weights, avoiding local maxima
Adagrad	scale each parameter's step	running s	parameters need different step sizes
+ Momentum	carry a running velocity	β	the gradient is noisy or bumpy
Adam	momentum + Adagrad	β_1, β_2	(industry default)